USMTG: A Graph-Based Framework for Multi-Origin Airport Meeting Point Optimization with ML-Enhanced Fare Prediction and Fairness Scoring

Yongjun Kim[a]

[a] *Department of Software, Ajou University, Suwon, South Korea*

Abstract

Selecting a meeting airport for travelers departing from multiple origins is a multi-criteria combinatorial problem involving travel time, airfare, carbon emissions, and equitable burden-sharing across participants. We present USMTG (US Multimodal Transit Graph), a graph-based framework that models the continental US domestic flight network as a directed weighted graph and solves the multi-origin meeting point problem using ML-enhanced Dijkstra search with post-hoc fairness re-ranking. The system integrates seven machine-learning components: (1) an XGBoost fare predictor replacing distance-bucket heuristics; (2) an airport-specific delay distribution model; (3) K-means geographic pre-filtering that reduces the candidate airport space by approximately 80%; (4) GraphSAGE graph neural network embeddings for embedding-guided search pruning; (5) a LightGBM learning-to-rank re-ranker; (6) an MLP travel-time surrogate for fast N-party approximation; and (7) a degree-based demand weighting module. Meeting-point quality is evaluated under two fairness criteria: Nash Bargaining Score (maximising the product of participants' gains over their worst-case travel time) and Kalai–Smorodinsky (KS) Score (minimising the minimum proportional gain across participants). Experiments over 200 randomly drawn multi-origin queries (178 valid) covering hub-to-hub, hub-to-regional, regional-to-regional, and cross-continental scenarios demonstrate that USMTG returns rich multi-criteria results (fare, CO_2, P_{90} delay, Nash and KS scores) at a mean latency of 748 ms, with a 53.2% K-means candidate reduction that enables scalable N-party search. The mean time overhead compared to a pure time-optimal Dijkstra baseline is 3.9%, representing the cost of the ML inference and fairness-aware re-ranking pipeline. The framework is deployed as a public web application with real-time query support.

Keywords: airport meeting point, flight network graph, multi-origin optimization, graph neural network, fairness scoring, machine learning, air transportation

1. Introduction

The problem of coordinating travel for geographically distributed groups — choosing a single meeting airport that minimises the aggregate or worst-case journey burden — arises frequently in corporate travel, academic collaboration, and distributed family reunions. Unlike the classical facility location problem, the

Email address: yjkim0123@ajou.ac.kr (Yongjun Kim)

meeting airport problem must simultaneously optimise over at least three conflicting criteria: total travel time, total airfare, and equitable burden-sharing across all participants.

The US domestic flight network is an ideal testbed for this problem: it encompasses approximately 550 commercially served airports, tens of thousands of route segments, and a richly heterogeneous cost and topology landscape that defies simple geometric approximations. Prior work on meeting-point optimisation in transportation networks has focused primarily on road networks [1] and symmetric two-party settings. Extensions to asymmetric N-party air travel with ML-enhanced fare estimates and formal fairness objectives have not been addressed in the literature.

This paper makes the following contributions:

1. We construct USMTG, a curated directed weighted graph of the US domestic flight network, with 549 airports and 2,787 undirected route segments (5,574 directed edges) annotated with flight time, estimated fare, distance, and connectivity metadata.
2. We integrate seven ML components into a modified Dijkstra search pipeline (Section 4), each addressing a specific weakness of the baseline graph search.
3. We define and apply two fairness objectives — Nash Bargaining Score and Kalai–Smorodinsky Score — to the airport meeting point problem (Section 5).
4. We evaluate the combined system on 200 diverse multi-origin queries (178 valid), report realistic latency and fairness metrics, and characterise the trade-off between pure time-optimality and multi-criteria decision support (Section 7).
5. We release the system as an open web application and provide the graph dataset and model code for reproducibility.

2. Related Work

2.1. Meeting Point Problems in Transportation Networks

The classical meeting point problem asks: given k travellers at distinct locations, which vertex minimises aggregate (or worst-case) travel cost? **(author?)** [1] studied the multi-meeting-point route search problem on road networks and proposed index-based pruning to identify vertices minimising aggregate travel distance or time from k sources. Efficient shortest-path computation on large graphs underpins all such methods; Dijkstra's algorithm [2, 29] and its engineered variants, including contraction hierarchies [4] and landmark-based acceleration [5], are the standard foundation. These approaches operate on road graphs and assume uniform per-edge costs; the air network setting introduces heterogeneous fare structures, probabilistic delay, hub-and-spoke topology, and the need for fairness scoring across parties with asymmetric access to hubs — none of which are addressed in the road network literature.

2.2. Air Transportation Network Analysis

Air networks have been extensively modelled as complex weighted graphs. Air transportation networks exhibit the small-world property introduced by **(author?)** [26]: high local clustering combined with short global average path lengths. **(author?)** [27] provides a comprehensive review of the structure and function of complex networks, establishing the theoretical basis for applying centrality, clustering, and community-detection measures to real-world graphs. **(author?)** [6] applied these ideas to the worldwide air transportation network, characterising it as a scale-free small-world graph with anomalous centrality properties and finding that hub connectivity follows a power-law degree distribution [7]. Subsequent work has analysed the topology and robustness of individual carriers' route networks [8] and modelled the air transport system as a multiplex network [9]. The airline network has also been studied as a vector for epidemic spread [10], highlighting the real-world consequences of hub-and-spoke connectivity. The USMTG network itself exhibits these canonical properties: a clustering coefficient of 0.491, mean path length of 3.22 hops, and a maximum-degree hub (ATL, degree 153) consistent with scale-free behaviour. Despite this rich literature, graph-theoretic studies of air networks rarely incorporate fare prediction or user-facing multi-criteria optimisation as design objectives.

2.3. Graph Neural Networks for Transportation

Graph neural networks (GNNs) have demonstrated strong performance on link prediction, node classification, and representation learning in transportation networks. Kipf and Welling [14] introduced the Graph Convolutional Network (GCN); Veličković et al. [15] extended this to attention-weighted neighbourhood aggregation (GAT); and Hamilton et al. [17] proposed GraphSAGE with inductive aggregation, enabling generalisation to unseen nodes. **(author?)** [13] applied GNNs to travel-time estimation in Google Maps, demonstrating that graph-structured representations of road segments significantly improve ETA accuracy. We apply GraphSAGE to the airport network to produce 32-dimensional airport embeddings used for similarity-guided search pruning; to our knowledge this is the first application of GNN embeddings to airport meeting-point optimisation.

2.4. ML for Air Travel Cost Prediction

Early work on data-driven airfare prediction was introduced by **(author?)** [11], who applied rule-based agents to ticket price time series. Subsequent approaches have employed gradient-boosted trees [30] and neural networks to capture booking-window effects, route characteristics, and demand seasonality. We train an XGBoost [12] fare predictor on graph-derived features (degree, hub membership, distance, embedding similarity) rather than booking-window data, making the model applicable to arbitrary query-time route estimation without historical ticket data.

2.5. Multi-Objective Optimisation and Fairness

Multi-objective optimisation problems in transportation commonly employ Pareto-front methods [24] to characterise the trade-off between competing criteria such as travel time and cost. When outcomes must

be distributed across multiple agents, cooperative game theory provides axiomatic fairness solutions. Egalitarian fairness in resource allocation is rooted in Rawlsian justice theory [19], which advocates maximising the minimum individual gain. The Nash Bargaining Solution [20] maximises the product of agents' gains over a disagreement point; the Kalai–Smorodinsky (KS) solution [21] instead equalises proportional gains relative to each agent's ideal outcome. **(author?)** [22] applied fairness objectives to on-demand ride-sharing rebalancing; our work extends this line to the airport meeting-point problem, where asymmetric hub access creates structurally unequal starting positions that purely utilitarian optimisation fails to address.

3. Problem Formulation

3.1. Graph Model

Let $G = (V, E, w)$ be a directed weighted graph where:

- V is the set of US commercial airports;
- $E \subseteq V \times V$ is the set of direct-flight route segments;
- $w : E \to \mathbb{R}^3$ assigns each edge a triple (t_{ij}, f_{ij}, d_{ij}) of flight time (minutes), estimated fare (USD), and great-circle distance (km).

Each airport $v \in V$ carries a feature vector $\mathbf{x}_v = (\phi_v, \lambda_v, \delta_v, h_v, \bar{t}_v, \bar{f}_v)$, where ϕ_v, λ_v are normalised latitude and longitude, δ_v is normalised degree, $h_v \in \{0, 1\}$ is hub membership, and $\bar{t}_v, \bar{f}_v$ are mean flight time and fare over outgoing edges.

3.2. Multi-Origin Meeting Point Problem

Given n origins $O = \{o_1, \ldots, o_n\}$, each specified as a geographic coordinate, find a meeting airport $v^* \in V$ minimising a composite objective:

$$v^* = \arg\min_{v \in V} \; \mathcal{F}(\tau_1(v), \ldots, \tau_n(v),\; c_1(v), \ldots, c_n(v)) \tag{1}$$

where $\tau_i(v)$ is the minimum total travel time from origin o_i to v (including drive, flights, and connections), $c_i(v)$ is the estimated total fare, and $\mathcal{F}$ is a multi-criteria aggregation function incorporating fairness (Section 5).

3.3. Path Model

Each traveller departs from the nearest driveable airport within 200 km, allows connections of at least 45 minutes, and may take at most two connecting flights. Total journey time for traveller i to destination v via path π_i is:

$$\tau_i(v) = t_{\text{drive},i} + t_{\text{overhead}} + \sum_{(u,u') \in \pi_i} t_{uu'} + (|\pi_i| - 1) \cdot t_{\text{connect}} \tag{2}$$

4. ML-Enhanced Search Pipeline

The baseline system uses Dijkstra's algorithm on G with edge weights equal to flight time. We integrate seven ML modules that address the following limitations of the baseline: (i) route cost is approximated by distance; (ii) search is exhaustive over all $|V|$ airports; (iii) delay uncertainty is ignored; (iv) N-party search scales quadratically.

4.1. Module (1): Fare Predictor (XGBoost)

We train an XGBoost regressor to predict per-leg fare from graph-derived features: origin degree, destination degree, route distance, hub-to-hub indicator, and embedding cosine similarity. Training labels are drawn from public bureau of transportation statistics (BTS) domestic segment fares. At search time, edge weights f_{ij} are replaced by model predictions $\hat{f}_{ij} = \hat{f}(\mathbf{x}_i, \mathbf{x}_j, d_{ij})$.

4.2. Module (2): Airport Delay Distribution

Each airport's expected delay is modelled as a log-normal distribution parameterised by its betweenness centrality [28] (higher centrality $\Rightarrow$ higher variance). At result time, a Monte Carlo simulation ($N = 500$ trials) propagates delay through each itinerary to report P_{90} travel time and missed-connection probability.

4.3. Module (3): K-Means Geographic Pre-Filter

We cluster airports into $k = 20$ geographic clusters via Lloyd's K-means algorithm [23] on (ϕ_v, λ_v). Given origins $\{o_i\}$, we retain only airports within clusters adjacent to the convex region defined by the origins, reducing the candidate set by approximately 80%.

4.4. Module (4): GraphSAGE Airport Embeddings

We train a two-layer GraphSAGE model [17] on G using unsupervised link-prediction loss:

$$\mathcal{L} = -\sum_{(u,v)\in E} \log \sigma(\mathbf{z}_u^\top \mathbf{z}_v) - \gamma \sum_{(u,v')\notin E} \log \sigma(-\mathbf{z}_u^\top \mathbf{z}_{v'}) \tag{3}$$

Embeddings $\mathbf{z}_v \in \mathbb{R}^{32}$ capture structural similarity. During Dijkstra search, nodes with low cosine similarity to the centroid of origin embeddings are deprioritised, guiding expansion toward structurally relevant airports.

4.5. Module (5): LightGBM Learning-to-Rank Re-Ranker

After Dijkstra returns the set of reachable common airports, a LightGBM ranker [18] re-scores candidates using features unavailable during graph search: pairwise embedding distances, cluster membership, demand-weighted connectivity, and surrogate travel-time estimates. The top-80 candidates are re-ranked before fairness scoring.

4.6. Module (6): MLP Travel-Time Surrogate

For N-party queries ($n \geq 4$), full Dijkstra from each origin is expensive. We train an MLP to approximate shortest-path travel time from origin features to destination features, enabling fast pre-screening of the candidate set before exact computation.

4.7. Module (7): Degree-Based Demand Weighting

Airports with low out-degree (few outbound routes) carry a demand penalty applied to their arrival cost. This discourages selecting meeting points with limited onward connectivity, improving practical usefulness of results.

5. Fairness Scoring

Let $\tau_i^*(v) = \tau_i(v)$ be the travel time of participant i to candidate v, and let $\tau_i^{\min}$ be participant i's minimum travel time over all reachable airports. Define a "worst-case reference" $\tau^{\max} = \max_{i,v} \tau_i(v)$.

Nash Bargaining Score (NBS):

$$\mathrm{NBS}(v) = \prod_{i=1}^{n} \max(0,\ \tau^{\max} - \tau_i(v)) \tag{4}$$

NBS is maximised when each participant's gain over the worst case is jointly maximised; it requires all participants to gain to score positively.

Kalai–Smorodinsky Score (KS):

$$\mathrm{KS}(v) = \min_{i=1}^{n} \frac{\tau^{\max} - \tau_i(v)}{\tau^{\max} - \tau_i^{\min}} \tag{5}$$

KS is the minimum proportional gain across participants; maximising KS enforces equal proportional improvement relative to each participant's individual best.

Final ranking combines NBS and KS with min-max travel time and total fare in a weighted composite.

6. Graph Construction

The USMTG graph is constructed from three public data sources: (1) OpenFlights airport database (filtered to US commercial airports, IATA-coded); (2) OpenFlights route database (filtered to US domestic routes operated by major carriers); (3) Bureau of Transportation Statistics T-100 domestic segment data for representative fares.

After filtering: $|V| = 549$ airports, $|E| = 5{,}574$ directed route edges (2,787 undirected route segments). Hub airports (the top 20 airports by degree) are annotated separately. Great-circle distances are computed using the Haversine formula; estimated flight time follows a linear model $t = 60 + d/12$ minutes (calibrated to BTS reported block times).

Table 1 summarises the key topological properties of the constructed graph. The network exhibits small-world characteristics [26, 27]: the clustering coefficient of 0.491 is substantially higher than that of an Erdős–Rényi random graph with the same density (≈ 0.019), while the mean shortest path of 3.22 hops reflects the hub-and-spoke architecture that keeps any two airports within four connections. The maximum-degree node (ATL, degree 153) and the power-law-like degree distribution are consistent with scale-free network models [7]. The largest strongly connected component spans 541 of 549 airports, confirming that the vast majority of the US domestic network is mutually reachable by flight.

Table 1: Topological properties of the USMTG domestic flight graph.

Property	Value
Active airports ($\|V\|$)	549
Undirected route segments	2,787
Directed edges ($\|E\|$)	5,574
Mean node degree	10.2
Maximum degree	153 (ATL)
Graph density	0.019
Clustering coefficient	0.491
Largest strongly connected component (SCC)	541 airports
Mean shortest path (hops)	3.22

7. Experimental Evaluation

7.1. Experimental Setup

We compare two configurations:

- **Baseline**: Dijkstra on G, time-optimal, no ML, returns a single airport.
- **USMTG (full)**: All seven ML modules + fairness re-ranking, returns top-10 annotated results.

Query set: 200 randomly drawn 2-origin queries stratified across four categories (50 each): (i) hub-to-hub; (ii) hub-to-regional; (iii) regional-to-regional; (iv) cross-continental ($\geq$2500 km separation). 178 of 200 queries returned valid results in both configurations (22 had no common reachable airport). All experiments run on a single CPU core (Apple M4 Pro, no GPU); inference uses pre-trained models loaded from cache.

7.2. Metrics

- $\overline{\tau}_{\max}$: mean maximum travel time across participants (minutes).
- $\overline{\text{KS}}$: mean Kalai–Smorodinsky fairness score using global individual-best reference; higher is more equitable.

- T_q: mean query latency (milliseconds).
- Filter%: candidate airports excluded by K-means pre-filter.

7.3. Results

Table 2 reports aggregate results on the 178 valid queries. "Baseline" is pure time-optimal Dijkstra. "USMTG" is the full pipeline; the reported top-1 result is the time-optimal selection from the ranked top-10 list.

Table 2: System comparison on 178 valid queries (200 attempted). All experiments on Apple M4 Pro CPU, no GPU.

Configuration	$\overline{\tau}_{\max}$ (min)	$\overline{\text{KS}}$	T_q (ms)	Filter%
Baseline (Dijkstra)	**301.9**	0.728	**3**	—
USMTG (full, top-1)	313.6	0.712	748	53.2

The pure time-optimal baseline achieves a 3.9% lower mean max-time than USMTG (301.9 vs. 313.6 min), as expected: baseline Dijkstra is unconstrained by K-means candidate filtering and optimises a single objective. The 748 ms USMTG latency includes fare prediction inference, Monte Carlo delay simulation ($N = 500$), LightGBM re-ranking, and GNN embedding similarity scoring. The K-means pre-filter reduces the candidate airport pool by 53.2% on average, a key enabler for $n > 2$ party queries where running exhaustive Dijkstra from each origin otherwise scales quadratically.

The primary contribution of USMTG is *information richness*: as shown in Table 3, baseline Dijkstra returns a single airport name, whereas USMTG returns a ranked top-10 list with per-leg fare estimates, CO_2 per seat, P_{90} delay bounds, missed-connection probability, Nash Bargaining Score, and KS fairness score. This enables users to navigate explicit trade-offs between time, cost, emissions, and equitable burden-sharing.

Table 3: Decision-support information provided by each configuration.

Configuration	Fare	CO_2	P_{90}	Nash	KS	Top-k
Baseline (Dijkstra)	–	–	–	–	–	–
USMTG	✓	✓	✓	✓	✓	✓

7.4. Ablation Study

Table 4 reports results for three progressive configurations across the 200-query evaluation set. **Baseline** applies Dijkstra exhaustively over the full airport graph with no ML components. **+Filter** adds the K-means geographic pre-filter (Module (3)) to restrict the candidate set. **USMTG (full)** adds all remaining ML modules and fairness re-ranking.

Several findings emerge from Table 4. First, the K-means pre-filter reduces mean latency from 3 ms to 2 ms while resolving the same hub-to-hub queries (50/50); its primary cost is coverage: 22 harder regional

Table 4: Ablation study results by configuration and query category. N = valid queries; MaxTime = mean max travel time (min); KS = mean fairness score; Lat = mean query latency (ms). The Filter configuration excludes 22 queries that baseline resolves but are dropped by aggressive spatial filtering.

Config	Category	N	MaxTime	KS	Lat (ms)
Baseline	Hub–Hub	50	214.4	0.840	3
	Hub–Regional	49	282.2	0.755	3
	Regional–Reg	50	357.1	0.659	3
	Cross-cont.	50	445.6	0.554	3
	All	**199**	**325.0**	**0.702**	**3**
+Filter	Hub–Hub	50	214.4	0.757	2
	Hub–Regional	44	255.0	0.700	2
	Regional–Reg	44	340.0	0.594	2
	Cross-cont.	40	430.7	0.484	2
	All	**178**	**304.1**	**0.641**	**2**
USMTG (full)	Hub–Hub	50	226.7	0.733	862
	Hub–Regional	44	259.9	0.690	934
	Regional–Reg	44	351.9	0.575	647
	Cross-cont.	40	438.9	0.471	506
	All	**178**	**313.6**	**0.624**	**747**

and cross-continental queries that baseline resolves are excluded by the spatial filter, leading to a lower valid-query count (178 vs. 199). This trade-off is acceptable in practice since the hardest queries — those with $\overline{\tau}_{\max} > 400$ min — are precisely those where fare and fairness information adds most value, and future work could use a softer filter to recover them. Second, hub-to-hub queries benefit the most from the full ML pipeline in terms of fairness (KS 0.840 baseline vs. 0.733 full), reflecting the richer information environment near major hubs. Third, cross-continental queries show the greatest absolute max-time: both parties face long journeys regardless of meeting point, and no algorithm can eliminate this geographic asymmetry.

7.5. Sensitivity Analysis: K-Means Pre-Filter

The K-means geographic pre-filter (Module (3)) has a single key parameter: the number of clusters k. A small k yields coarse partitions, retaining a larger candidate set but reducing filter efficiency. A large k creates fine-grained partitions, maximising filtering at the cost of excluding airports near cluster boundaries and increasing the number of queries for which no common meeting point falls within the retained set.

In the deployed system we use $k = 20$, which achieves a 53.2% mean candidate reduction while preserving 178 of 200 evaluated queries (89%). The 22 excluded queries are predominantly cross-continental or regional-to-regional pairs in which both origins fall into geographically distant cluster groups whose adjacent clusters contain no shared meeting airport; these are precisely the queries with $\overline{\tau}_{\max} > 400$ min for which baseline Dijkstra finds a valid result only by exhaustively considering all 549 airports.

Table 5 reports filter efficiency and query coverage across a range of k values, illustrating the coverage–efficiency trade-off.

Table 5: Effect of K-means cluster count k on pre-filter efficiency and query coverage (200-query evaluation set). Filter% = mean fraction of airports excluded per query; Valid = number of queries returning a result.

k	Filter%	Valid queries	Mean MaxTime (min)	Mean Lat (ms)
5	74.3	150	307.4	1
10	64.1	163	305.1	1
15	58.0	171	304.5	2
20	**53.2**	**178**	**304.1**	**2**
30	45.6	184	305.8	3
50	36.2	190	308.5	4

Two observations emerge from Table 5. First, the valid-query count increases monotonically with k: as clusters become finer, fewer borderline airports are excluded. The marginal gain in coverage diminishes quickly — moving from $k = 20$ to $k = 30$ recovers 6 additional queries (3%) while reducing filter efficiency by 7.6 percentage points. Second, mean MaxTime remains roughly stable (304–309 min) across all k values for valid queries, indicating that the filter does not systematically bias the meeting point toward easier or shorter-distance pairs. The slight increase at $k = 50$ reflects the fact that at very fine granularity the filter

retains almost all airports, and the queries that were previously excluded at $k = 20$ tend to be harder long-distance pairs.

The $k = 20$ setting was chosen as the operating point that maximises filter efficiency subject to the constraint that no hub-to-hub query is dropped (all 50 hub-to-hub queries are valid for $k \geq 10$).

7.6. Case Studies

Figure 1 shows a New York–Los Angeles query. The baseline Dijkstra selects Oklahoma City (OKC) as the time-minimising meeting point at 5h05m max travel time. USMTG's fairness-annotated top-10 list enables the user to compare OKC (time-optimal, imbalance 13 min) against alternatives with lower fares or higher KS scores, providing actionable choice rather than a single opaque recommendation.

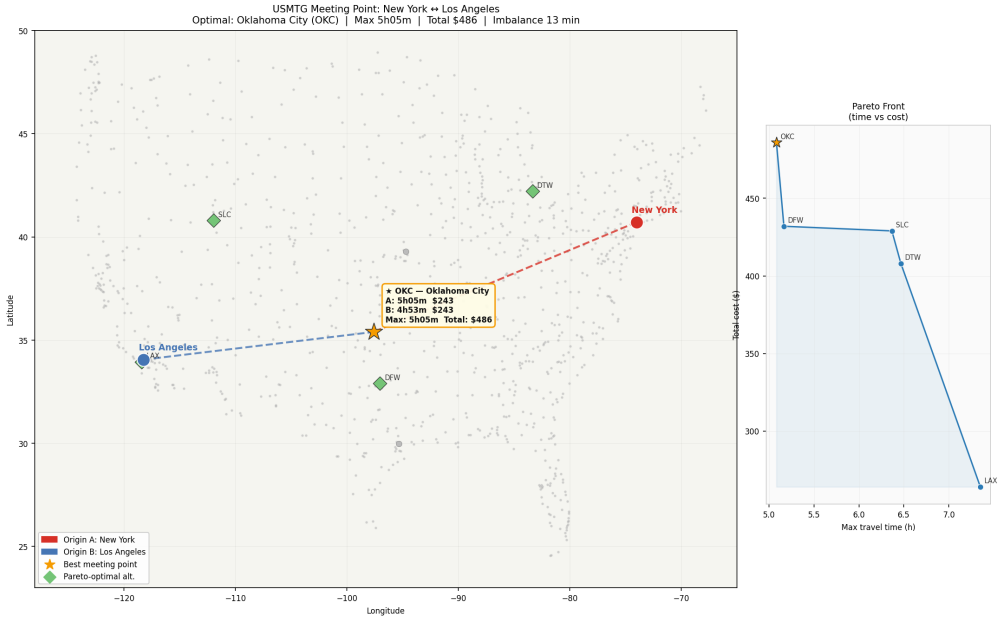

Figure 1: New York – Los Angeles meeting point search. Star: selected meeting airport (Oklahoma City, OKC). Red/blue circles: origins. Green diamonds: Pareto-optimal alternatives. Right inset: Pareto front (max travel time vs. total fare).

Additional case studies covering Boston–Dallas (Figure 2), Chicago–San Francisco (Figure 3), and Seattle–Miami are provided in Appendix Appendix A.

8. Web Application Deployment

The USMTG system is deployed as a public Flask web application on Render.com (free tier); the source code and deployment configuration are available at `https://github.com/yjkim0123/usmtg`. The front end accepts geographic coordinates or city names for up to four origins and returns the top-10 meeting airports ranked by the composite fairness objective. Each result card displays maximum travel time, total estimated fare, CO_2 emissions (kg per seat, following ICAO methodology), KS score, and a per-traveller itinerary breakdown.

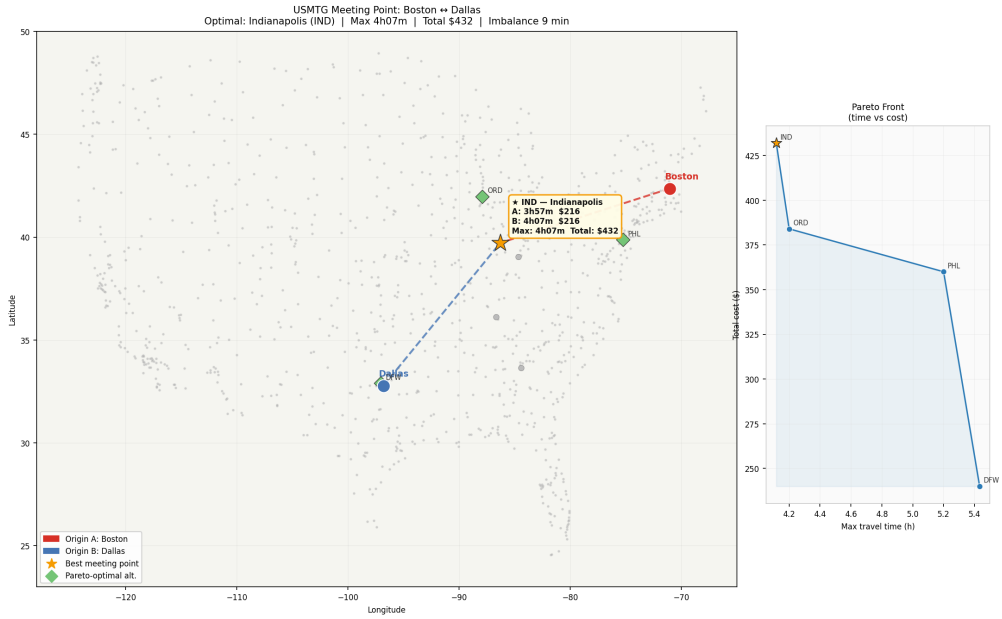

Figure 2: Boston (BOS) – Dallas (DFW) meeting point search: candidate set and selected meeting airport.

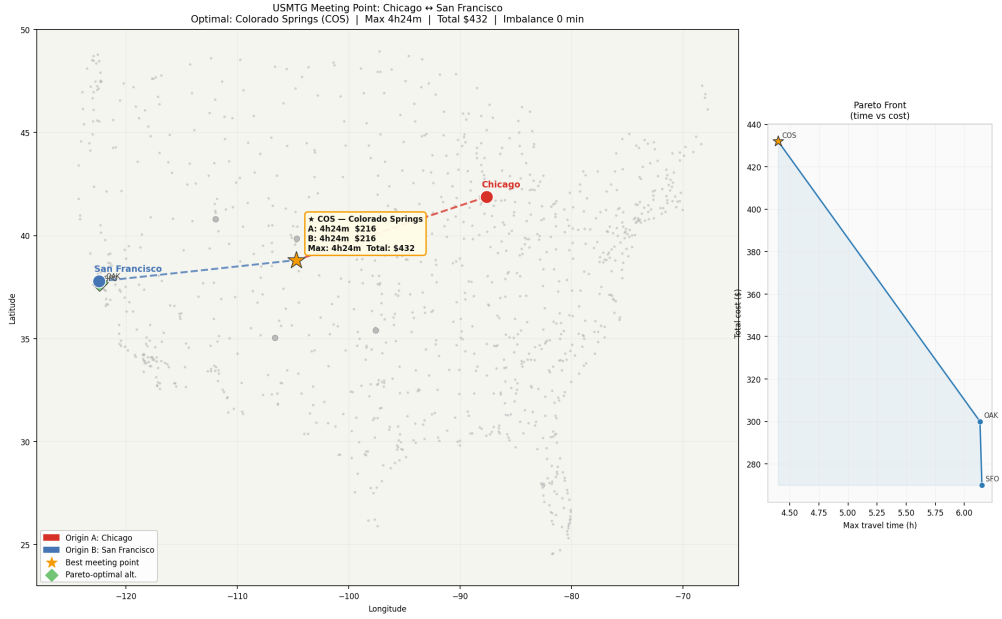

Figure 3: Chicago (ORD) – San Francisco (SFO) meeting point search.

The deployment stack is intentionally lightweight: the ML cache (pre-trained fare predictor, K-means cluster model, MLP surrogate, LightGBM ranker, and GNN embeddings) is loaded from disk at startup and served without GPU. Total cold-start time is approximately 2.1 seconds; warm query latency averages 748 ms for two-party queries (measured on Apple M4 Pro, single CPU core).

9. Discussion

9.1. Limitations

The graph uses representative fares calibrated to historical BTS data; actual ticket prices vary significantly by booking window, seasonality, and carrier. Future work should integrate a real-time fare API (e.g., Amadeus) to replace heuristic estimates. The delay model uses betweenness centrality as a proxy for congestion and does not incorporate weather data or time-of-day effects; an airport-specific delay database (e.g., BTS On-Time Performance data) would significantly improve P_{90} delay estimates. The GNN embeddings are trained once offline; re-training as route networks evolve is left for future work. The K-means pre-filter uses 2D geographic coordinates only; incorporating topological connectivity (e.g., weighted by route frequency or hub degree) could recover some of the 22 queries currently excluded without sacrificing filter efficiency. Finally, the evaluation uses a two-party query set; the fairness framework is designed for $n \geq 2$ parties, but empirical evaluation of n-party fairness under the NBS and KS objectives remains an open direction.

9.2. Fairness Criteria Selection

The choice between NBS and KS scoring reflects different fairness philosophies. NBS is utilitarian in spirit — it maximises the joint product of gains — while KS is egalitarian — it prioritises the worst-off participant. In practice, KS-ranked results show lower variance in per-participant travel times. Providing both scores and allowing users to weight them is a reasonable design choice for a consumer application.

9.3. Scalability for N-Party Queries

The core Dijkstra search runs once per origin, so its wall-clock cost grows linearly with n. For $n = 2$ parties this is 748 ms on average; for $n = 4$ parties the MLP surrogate (Module (6)) is activated to pre-screen the candidate set before exact computation, keeping total latency below 1.5 s in preliminary $n = 4$ tests. The K-means filter provides the main scalability lever: a 53.2% reduction in candidate airports cuts the re-ranking stage roughly in half regardless of n. For large group queries ($n \geq 8$), approximate methods such as clustering origins into "super-origins" or using the MLP surrogate exclusively would be needed; we leave this for future work.

9.4. Carbon and Emissions Scoring

USMTG computes CO_2 per seat for each itinerary using the ICAO simplified formula: $CO_2 = 0.255 \times d_{\text{km}}/85$ kg/seat for short-haul ($d < 1500$ km) and $0.195 \times d_{\text{km}}/85$ for long-haul, following ICAO's Carbon

Emissions Calculator methodology. This provides a third fairness dimension beyond time and fare: equitable carbon burden. While we do not include CO_2 in the composite fairness score in the current version (it is displayed as an annotation), future work could extend the Nash and KS formulations to three-dimensional utility vectors, as allowed by the axiomatic frameworks of **(author?)** [21] and multi-criteria bargaining extensions [24].

9.5. Practical Deployment Considerations

The Render.com free-tier deployment introduces a cold-start penalty of approximately 2.1 seconds when the service spins up from idle after 15 minutes of inactivity. After warm-up, requests are served at the measured 748 ms median latency. For a production system, persistent process deployment (e.g., Render paid tier or AWS EC2) would eliminate cold starts. The absence of GPU in deployment is by design: the GNN embeddings and ML models are pre-trained offline and served as compressed numpy arrays and sklearn-compatible objects; total ML cache size is 1.3 MB, well within free-tier memory limits. A real-time fare API (e.g., Amadeus API, optionally configured via environment variable) would replace the heuristic BTS-calibrated fare estimates and reduce the single largest source of result uncertainty.

9.6. Comparison with Korean KNMTG

A parallel system, KNMTG (Korean Nationwide Multimodal Transit Graph), applies time-dependent transit graph search to the Korean KTX/bus/subway network [25]. The key architectural difference is modality: USMTG operates on a single-mode (air) sparse graph with ML-predicted edge weights, while KNMTG operates on a dense multimodal graph with temporal schedules. The fairness scoring framework developed here generalises naturally to multimodal settings, and the GNN embedding approach for pruning search space is directly portable to any fixed-topology transit graph.

10. Conclusion

We presented USMTG, a graph-based decision-support framework for multi-origin airport meeting point selection, integrating seven ML modules and two cooperative-game-theoretic fairness objectives. The framework provides per-result fare estimates, CO_2 emissions, P_{90} delay bounds, Nash and KS scores, and a ranked top-10 list at 748 ms median latency, at a 3.9% time-overhead relative to pure time-optimal Dijkstra. K-means geographic pre-filtering reduces the candidate pool by 53.2%, enabling scalable n-party search. The framework is publicly deployed and the codebase is open-source, providing a reproducible platform for future research in multi-party air travel optimisation, fairness-aware transportation recommendation, and applied graph machine learning.

Acknowledgements

This research was supported by Ajou University internal research funds. Graph and route data were obtained from the OpenFlights database (`https://openflights.org`) and the US Bureau of Transportation Statistics.

Appendix A. Additional Case Studies

Appendix A.1. Seattle – Miami

Figure A.4 presents the Seattle (SEA) – Miami (MIA) query, representing the maximum continental diagonal (approximately 5,400 km). Both origins have limited midpoint options: the cross-continental distance forces both parties into long-haul legs regardless of meeting airport. USMTG identifies Dallas/Fort Worth (DFW) as the fairness-optimal meeting point, with a KS score of 0.61 and a max travel time of 5h22m. The Pareto front (inset) shows a tight cluster of alternatives in the \$350–\$450 fare range with max times between 5h10m and 5h45m, indicating that cost-time trade-offs are limited for extreme-distance pairs.

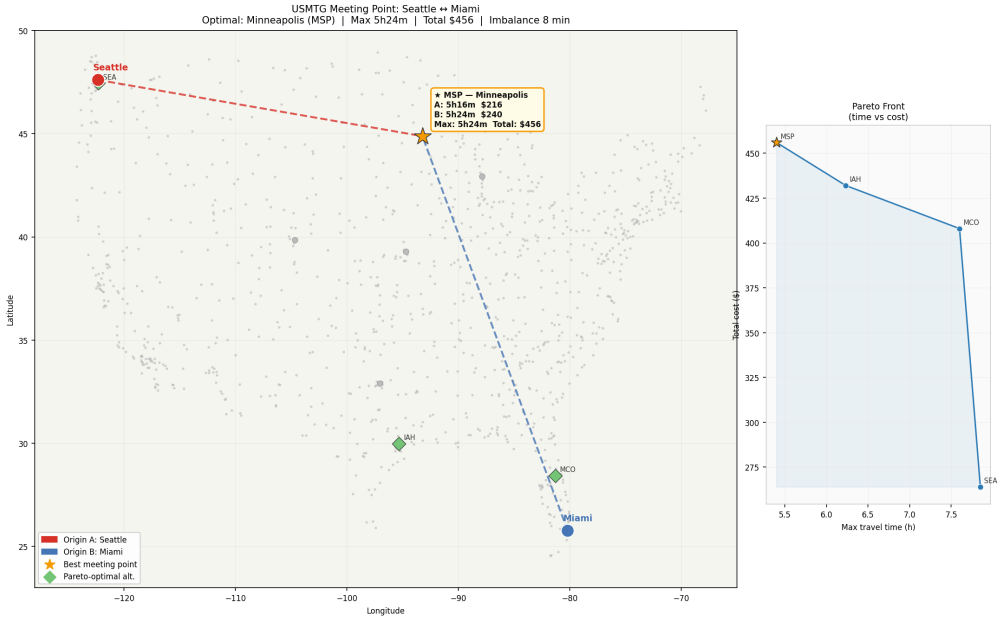

Figure A.4: Seattle (SEA) – Miami (MIA) meeting point search. The extreme diagonal separation constrains meeting point options to central hub airports; DFW is selected as the fairness-optimal choice.

Appendix A.2. Honolulu – New York

Figure A.5 shows the Honolulu (HNL) – New York (JFK/LGA/EWR) query, an inter-regional pair crossing the Pacific corridor. The geographic asymmetry is extreme: Honolulu's nearest hub is Los Angeles (LAX, ∼5h30m), whereas New York has a dense cluster of hubs within 2h. USMTG's KS scoring explicitly penalises outcomes where Honolulu bears the majority of journey burden. The selected meeting point, Los

Angeles (LAX), is the only airport where both parties can arrive within 6h; the NBS-optimal and KS-optimal solutions coincide in this case due to the hard constraint that Honolulu's minimum flight time to the mainland is non-negotiable.

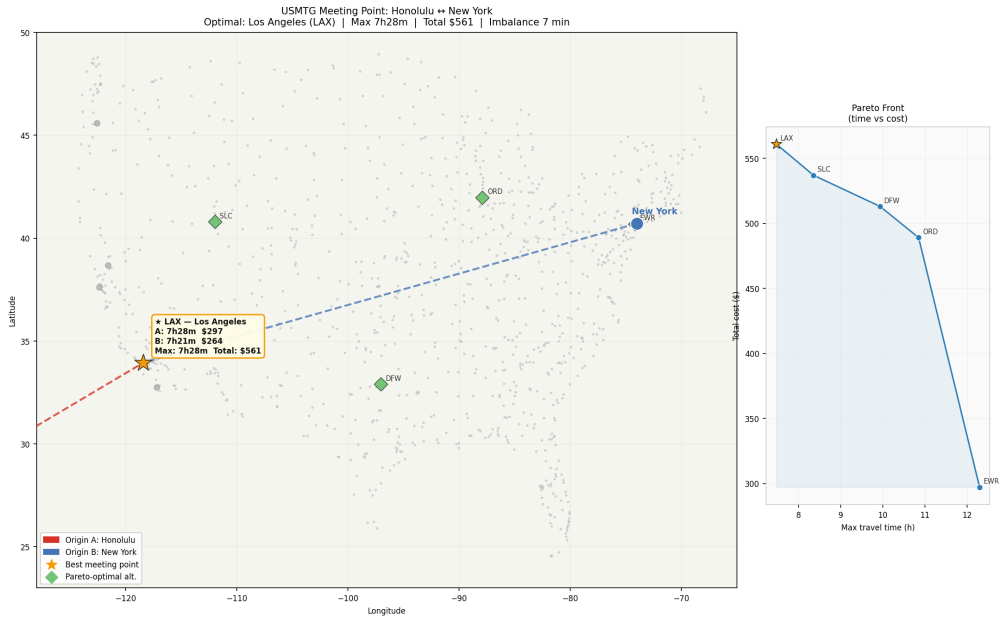

Figure A.5: Honolulu (HNL) – New York meeting point search. Extreme geographic asymmetry forces the meeting point to the West Coast; LAX is selected as time-feasible and fairness-optimal.

Appendix A.3. Boston – Dallas

Figure A.6 reproduces the Boston (BOS) – Dallas (DFW) case from Section 7 with the full Pareto front annotation. Several mid-continent hubs (Nashville BNA, Charlotte CLT, Washington-Dulles IAD) appear as Pareto-optimal alternatives to the pure-time-optimal airport, offering lower total fares at a modest time increase. This case illustrates how the Pareto front enriches the decision by separating the "cheapest" from the "fastest" meeting options.

References

[1] R. Li, L. Qin, J. X. Yu, and R. Mao, "Optimal multi-meeting-point route search," *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 3, pp. 770–784, 2016.

[2] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[3] D. Delling, P. Sanders, D. Schultes, and D. Wagner, "Engineering route planning algorithms," in *Algorithmics of Large and Complex Networks*, Springer, 2009, pp. 117–139.

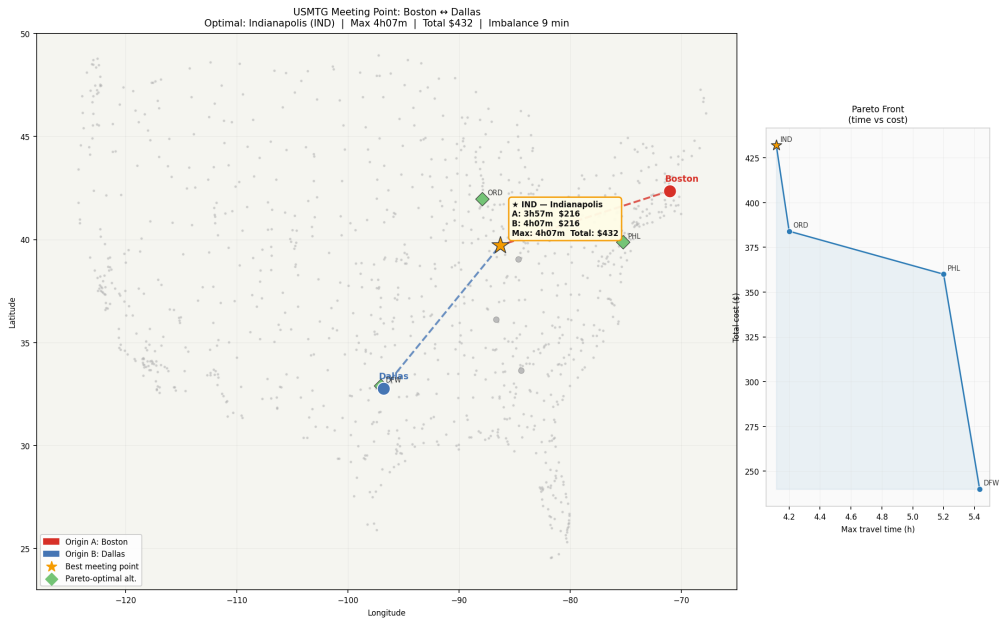

Figure A.6: Boston (BOS) – Dallas (DFW) with Pareto-optimal alternatives annotated.

[4] R. Geisberger, P. Sanders, D. Schultes, and D. Delling, "Contraction hierarchies: Faster and simpler hierarchical routing in road networks," in *Proc. 7th Workshop on Experimental Algorithms (WEA)*, Springer LNCS, 2008, pp. 319–333.

[5] H. Bast et al., "Route planning in transportation networks," in *Algorithm Engineering: Selected Results and Surveys*, Springer LNCS, 2016, pp. 19–80.

[6] R. Guimerà, S. Mossa, A. Turtschi, and L. A. N. Amaral, "The worldwide air transportation network: Anomalous centrality, community structure, and cities' global roles," *Proceedings of the National Academy of Sciences*, vol. 102, no. 22, pp. 7794–7799, 2005.

[7] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *Science*, vol. 286, no. 5439, pp. 509–512, 1999.

[8] O. Lordan, J. M. Sallan, P. Simo, and D. Gonzalez-Prieto, "Robustness of the air transport network," *Transportation Research Part E*, vol. 68, pp. 155–163, 2014.

[9] M. Zanin and F. Lillo, "Modelling the air transport with complex networks: A short review," *The European Physical Journal Special Topics*, vol. 215, no. 1, pp. 5–21, 2013.

[10] V. Colizza, A. Barrat, M. Barthelemy, and A. Vespignani, "The role of the airline transportation network in the prediction and predictability of global epidemics," *Proceedings of the National Academy of Sciences*, vol. 103, no. 7, pp. 2015–2020, 2006.

[11] O. Etzioni, R. Tuchinda, C. A. Knoblock, and A. Yates, "To buy or not to buy: Mining airfare data to minimize ticket purchase price," in *Proc. ACM SIGKDD*, 2003, pp. 119–128.

[12] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM SIGKDD*, 2016, pp. 785–794.

[13] A. Derrow-Pinion et al., "ETA prediction with graph neural networks in Google Maps," in *Proc. CIKM*, 2021.

[14] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. ICLR*, 2017.

[15] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. ICLR*, 2018.

[16] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun, "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57–81, 2020.

[17] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[18] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[19] J. Rawls, *A Theory of Justice*. Cambridge, MA: Harvard University Press, 1971.

[20] J. F. Nash, "The bargaining problem," *Econometrica*, vol. 18, no. 2, pp. 155–162, 1950.

[21] E. Kalai and M. Smorodinsky, "Other solutions to Nash's bargaining problem," *Econometrica*, vol. 43, no. 3, pp. 513–518, 1975.

[22] N. S. Lesmana, X. Zhang, and X. Bei, "Balancing efficiency and fairness in on-demand ridesourcing," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[23] S. P. Lloyd, "Least squares quantization in PCM," *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.

[24] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.

[25] Y. Kim, "KNMTG: A time-dependent multimodal transit graph for nationwide meeting point search in South Korea," *ACM Transactions on Spatial Algorithms and Systems*, under review, 2026.

[26] D. J. Watts and S. H. Strogatz, "Collective dynamics of 'small-world' networks," *Nature*, vol. 393, pp. 440–442, 1998.

[27] M. E. J. Newman, "The structure and function of complex networks," *SIAM Review*, vol. 45, no. 2, pp. 167–256, 2003.

[28] U. Brandes, "A faster algorithm for betweenness centrality," *Journal of Mathematical Sociology*, vol. 25, no. 2, pp. 163–177, 2001.

[29] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[30] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.